

1회차: 고급 위젯 및 복잡한 기능의 뷰 컨테이너 마스터 가이드

1. 날짜 및 시간 관련 위젯 계층 구조와 변천사

안드로이드의 시간 표시 위젯은 API 버전 업데이트에 따라 권장 사항이 크게 변경되었으므로 시험에 출제될 확률이 매우 높습니다.

1.1 시계 위젯 (`AnalogClock` & `DigitalClock`)

- 계층 구조 파악:

- `java.lang.Object` → `android.view.View` → `android.widget.AnalogClock`
- `java.lang.Object` → `android.view.View` → `android.widget.TextView` → `android.widget.DigitalClock`

- `DigitalClock` (사용 중단/Deprecated):

- API 17 (Android 4.2)부터 공식적으로 사용이 중단되었습니다.
- 최신 환경에서는 한층 업그레이드된 `TextClock` 위젯을 대신 사용하도록 권장합니다.

- `AnalogClock` (사용 비권장):

- 디자인이 구식이고 시각적 커스터마이징이 까다로워 현재는 개발 시 권장하지 않는 위젯입니다.
- ⚠ **시험 포인트:** 최신 안드로이드 스튜디오의 팔레트(Palette) 툴바는 개발 편의를 위해 **최신 위젯만 기본적으로 표시하며**, 오래된 위젯은 노출되지 않습니다.

1.2 `Chronometer` (크로노미터 타이머)

- 정의:** 타이머 형태의 위젯으로 초 단위의 경과 시간을 측정하고 화면에 노출할 때 필수적으로 사용됩니다.

- 핵심 제어 메서드 3종:

- `setBase(long base)` : 타이머가 계산을 시작할 기준점(0초 지점)을 설정합니다. 주로 시스템이 부팅된 후 경과된 시간인 `SystemClock.elapsedRealtime()` 을 넘겨 초기화합니다.
- `start()` : 시간 측정을 시작합니다.

- `stop()` : 시간 측정을 일시 중지합니다.

1.3 DatePicker & TimePicker

- 사용자에게 날짜(연, 월, 일)와 시간(시, 분)을 직관적으로 선택할 수 있는 팝업 인터페이스를 제공합니다.
- **출제 팁:** 기본 달력 형태가 아닌, 위아래로 굴러 선택하는 전통적인 스피너(Spinner) 스타일로 화면을 고정하려면 XML 속성에 반드시 `android:datepickerMode="spinner"` 또는 `android:timepickerMode="spinner"` 를 명시해야 합니다.

2. 특수 목적형 고급 위젯 핵심 요약

2.1 자동완성 텍스트뷰 계열

사용자가 타이핑을 시작할 때 등록된 데이터 어댑터(`ArrayAdapter`) 리스트를 검색하여 추천 단어를 하단에 드롭다운 형태로 띄워주는 편리한 입력 위젯입니다.

- **AutoCompleteTextView** :
 - 단 하나의 완성된 단어만 추천 및 자동 입력되도록 지원합니다.
- **MultiAutoCompleteTextView** :
 - 여러 개의 단어를 연속해서 자동완성할 수 있습니다.
 - **핵심 메커니즘:** 단어와 단어를 논리적으로 구분해 주는 **토큰라이저(Tokenizer)** 설정이 반드시 필요합니다. 코드단에서 `setTokenizer(new MultiAutoCompleteTextView.CommaTokenizer())` 를 선언하면 쉼표(,)를 기준으로 연속적인 자동완성을 지원합니다.

2.2 진척도 및 평가 위젯 (`ProgressBar` , `SeekBar` , `RatingBar`)

- **ProgressBar** : 작업의 진행 상태를 막대(Bar)나 회전하는 원형(Circle) 형태로 제공합니다.
- **SeekBar** : 프로그레스바의 개념에 사용자가 터치하여 조절할 수 있는 패드(Thumb)를 결합한 위젯입니다. 드래그 이벤트를 감지하기 위해 `OnSeekBarChangeListener` 인터페이스를 구현합니다.
- **RatingBar** : 만족도나 평점을 **별 모양 점수**로 시각화합니다.
 - **중요 속성:** `android:stepSize="0.5"` (별점이 변화하는 최소 단위 설정), `android:numStars="5"` (전체 별 개수 제한).

3. 복잡한 구조의 뷰 컨테이너 (View Containers)

레이아웃 내부에 다른 위젯들을 효율적으로 배치하고, 화면 전환이나 스크롤 등의 역동적인 기능을 부여하는 특수 뷰그룹입니다.

3.1 **ScrollView** (스크롤뷰)

- 화면 크기를 초과하는 긴 콘텐츠가 존재할 때 상하 스크롤을 만들어 화면 이탈을 방지합니다.
- **⚠ 절대적인 규칙 (시험 단골):** 스크롤뷰 레이아웃 바로 밑에는 반드시 단 하나의 자식 뷰(Single Child)만 배치해야 합니다. 만약 여러 위젯을 직접 나열하면 빌드 에러가 나므로, 보통 대형 `LinearLayout` 하나를 두고 그 내부에 다양한 위젯들을 집어넣는 구조로 설계합니다. (좌우 스크롤은 `HorizontalScrollView` 사용)

3.2 **ViewFlipper** (뷰플리퍼)

- 내부에 여러 개의 위젯이나 레이아웃을 겹쳐서 배치한 다음, 조건에 맞춰 한 번에 단 하나의 화면만 번갈아가며 보여주는 컨테이너입니다.
- **주요 메서드:**
 - `showNext()` : 다음 화면으로 전환
 - `showPrevious()` : 이전 화면으로 전환
 - `startFlipping()` / `stopFlipping()` : 자동 화면 전환 시작 및 종료
 - `setFlipInterval(int ms)` : 자동 전환될 주기를 밀리초(ms) 단위로 지정

3.3 **TabHost** (탭호스트)와 시스템 예약 ID의 비밀

- 상단이나 하단에 탭 버튼을 배치해 한 화면에서 여러 테마의 페이지를 슬라이드 분할 전환하는 컨테이너입니다.
- **⚠ 뼈대 구조 예약 ID 암기:** 안드로이드 OS 아키텍처가 탭의 구성 요소를 안전하게 로드할 수 있도록 XML 작성 시 아래 시스템 지정 ID를 **토씨 하나 틀리지 않고 그대로 적용**해야 정상 구동됩니다.
 1. `TabHost` 위젯 ID → `@android:id/tabhost`
 2. 탭 버튼 모음인 `TabWidget` ID → `@android:id/tabs`
 3. 페이지 내용물이 담기는 `FrameLayout` ID → `@android:id/tabcontent`

3.4 **WebView** (웹뷰)와 간단 웹브라우저 실습

- 앱 내부 화면에 실제 외부 인터넷 웹페이지(`http://...`)를 통째로 렌더링하여 보여주는 고성능 위젯입니다.

간단 웹브라우저 구현 자바 베스트 프랙티스 코드

Java

```
package com.exam.androidstudio;

import android.os.Bundle;
import android.view.View;
import android.webkit.WebSettings;
import android.webkit.WebView;
import android.webkit.WebViewClient;
import android.widget.Button;
import android.widget.EditText;
import androidx.appcompat.app.AppCompatActivity;

public class WebBrowserActivity extends AppCompatActivity{

    private EditText edtUrl;
    private Button btnGo, btnBack;
    private WebView web;

    @Override
    protected void onCreate(Bundle savedInstanceState){
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_web_browser);

        // UI 위젯 객체 바인딩
        edtUrl = findViewById(R.id.edtUrl1);
        btnGo = findViewById(R.id.btnGo1);
        btnBack = findViewById(R.id.btnBack1);
        web = findViewById(R.id.webView1);

        // [중요] 내장 웹뷰 클라이언트 클래스 지정 (새 창으로 브라우저가 열리는 현상 방지)
        web.setWebViewClient(new CustomWebViewClient());

        // 웹뷰의 세부 속성 제어를 위한 WebSettings 객체 획득
        WebSettings webSet = web.getSettings();
        webSet.setBuiltInZoomControls(true); // 화면 확대/축소 아이콘 레이아웃 활성화
        webSet.setJavaScriptEnabled(true); // 웹페이지 내 자바스크립트 실행 허용

        // 이동 버튼 클릭 이벤트 리스너 설정
        btnGo.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v){
                // 사용자가 입력한 URL 문자열을 읽어와 웹뷰에 로드
                String url = edtUrl.getText().toString();
                web.loadUrl(url);
            }
        });

        // 이전 페이지 가기 버튼 리스너 설정
        btnBack.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v){
```

```

        // 웹뷰 히스토리상 이전 페이지가 존재하면 뒤로 가기 수행
        if (web.canGoBack()) {
            web.goBack();
        }
    }
});
}

/**
 * 웹뷰 내에서의 페이지 로딩 및 이벤트를 전담 제어하는 내부 클래스
 */
private static class CustomWebViewClient extends WebViewClient{
    @Override
    public boolean shouldOverrideUrlLoading(WebView view, String url){
        // 외부 브라우저를 켜지 않고 현재 웹뷰 내부에서 페이지가 계속 열리도록 선언
        view.loadUrl(url);
        return true;
    }
}
}

```